Ordenamiento por Intercambio (Bubble Sort), del juego de tarjetas al código

Objetivo

Comprender el algoritmo de **Bubble Sort** mediante la simulación con tarjetas. Después, diseñar una solución en **pseudocódigo** e implementarla en **Java**.

Materiales

- Tarjetas con números o nombres (mínimo 5). Ejemplo: 34, 12, 25, 8, 19.
- Hojas para registrar las pasadas.
- Computadora con JDK instalado y Bloc de Notas

Fase 1: Simulación con tarjetas (se conserva de la actividad original)

1. Formen equipos de 5 o más personas. Cada integrante es una tarjeta y se colocan en fila.
2. Comparen pares adyacentes de izquierda a derecha. Si la tarjeta de la izquierda es mayor que la de la derecha, intercambian lugares.
3. Recorrer toda la fila es **una pasada**. Repitan las pasadas hasta que ya no haya intercambios.
4. **Registren en una tabla** el estado de la fila al terminar cada pasada, el número de comparaciones y el número de intercambios.

Fase 2: Propuesta de solución en pseudocódigo (en binas)

A partir del registro de la Fase 1, escriban su propio pseudocódigo. Debe responder a estas preguntas:

- ¿Qué estructura de datos representa la fila de tarjetas?
  R= Se representa por un arreglo

- ¿Qué ciclo representa las **pasadas** y cuál representa las **comparaciones** dentro de una pasada?
  R= el cliclo que representa las pasadas es el for y el que las compara es un if

- ¿Cómo se realiza el **intercambio** de dos elementos sin perder uno de ellos?
  R= se utiliza una variable temporal como auxiliar que se use momentáneamente para hacer el intercambio.

- ¿Cómo sabe el algoritmo que **ya terminó**, es decir, cuándo detenerse antes de completar todas las pasadas?
  R= Cuando se hace el recorrido y en todas las compraciones y al querer hacer el intercambio en la sentencia el resultado es falso
- ¿Cuántos elementos hay que comparar en cada pasada? Piensen en qué tarjetas ya quedaron en su lugar definitivo.
  R= En cada pasada se realizan menos comparaciones, porque el elemento más grande va quedando en su posición definitiva al final del arreglo.

José Emmanuel Arredondo Escalante

Entregable: el pseudocódigo en hoja o archivo, con nombres de variables claros y sangría.

Fase 3: Implementación en Java

1. Traduzcan su pseudocódigo a un programa en Java con un método bubbleSort(int[] arreglo).
2. El programa debe mostrar el arreglo **después de cada pasada**, para compararlo con el registro de la Fase 1.
3. Debe contar e imprimir el **total de comparaciones, intercambios y pasadas**.
4. Prueben con el arreglo {34, 12, 25, 8, 19}, con uno ya ordenado y con uno en orden inverso.
5. Variación opcional: ordenar un arreglo de String alfabéticamente con compareTo.

Pseucodigo:

```
Algoritmo hola
   Definir lista, i, z, a Como Entero
   Dimension lista[5]

   lista[0] <- 3
   lista[1] <- 7
   lista[2] <- 10
   lista[3] <- 17
   lista[4] <- 9

   Para i <- 0 Hasta 3 Con Paso 1 Hacer
      Para z <- 0 Hasta 3 Con Paso 1 Hacer
         Si lista[z] > lista[z+1] Entonces
            a <- lista[z]
            lista[z] <- lista[z+1]
            lista[z+1] <- a
         FinSi
      FinPara
   FinPara

   Para i <- 0 Hasta 4 Con Paso 1 Hacer
      Escribir lista[i]
   FinPara

FinAlgoritmo
```

Java:

José Emmanuel Arredondo Escalante

```
import java.util.Arrays;

public class Burbuja {

    public void bubleS(int[] arreglo) {

        int aux;
        int comparaciones = 0;
        int intercambios = 0;

        for (int i = 0; i < arreglo.length - 1; i++) {
            for (int z = 0; z < arreglo.length - 1; z++) {
                comparaciones=comparaciones+1;
                if (arreglo[z] > arreglo[z + 1]) {
                    aux = arreglo[z];
                    arreglo[z] = arreglo[z + 1];
                    arreglo[z + 1] = aux;
                    intercambios = intercambios+1;;
                }
            }
            System.out.println("Pasada " + (i + 1)+ Arrays.toString(arreglo));
        }
        System.out.println("Comparaciones: " + comparaciones);
        System.out.println("Intercambios: " + intercambios);
        System.out.println("Pasadas: " + (arreglo.length - 1));
    }
    public static void main(String[] args) {
         Burbuja objeto = new Burbuja();

        int[] lista = {34, 12, 25, 8, 19};

        objeto.bubleS(lista);
    }
}
```

José Emmanuel Arredondo Escalante

Fase 4: Reflexión

- ¿Coincide la salida del programa con lo que registraron con las tarjetas?
  R= es el mismo procedimiento
- ¿Qué cambios tuvo que hacer su pseudocódigo al pasarlo a Java?
  R= La estructura y sintaxis, así como la forma en la que se añaden los arreglos y también el cómo se ve el tamaño del arreglo, así en el for se puede ciclar de manera dinámica, dependiendo del tamaño del arreglo y así ya no es fijo.
- ¿Qué pasa cuando el arreglo ya está ordenado? ¿Sigue comparando?
  R= si hace las comparaciones que están determinadas al tamaño del arreglo ya que asi por si solo no hay forma de determinar si están ordenados sin hacer el recorrido.
- ¿Es eficiente con listas muy grandes? Relacionen su respuesta con el número de comparaciones ($O(n^2)$).
  R= no es eficiente ya que hace el número de comparaciones dependiendo del tamaño del arreglo, lo que en grandes arreglos hace que sea una mayor numero de comparaciones lo que hace que sea ineficiente.

Lista de cotejo (5 puntos)

#	Criterio	Cumple	No cumple	Observaciones
1	**Simulación y registro:** la tabla de pasadas con tarjetas es completa y correcta (estado de la fila, comparaciones e intercambios por pasada).			
2	**Pseudocódigo:** es claro, está estructurado y contiene los dos ciclos (pasadas y comparaciones), la condición de comparación, el intercambio con variable auxiliar y la condición de paro.			
3	**Implementación en Java:** el programa compila, ejecuta y ordena correctamente el arreglo de ejemplo y los casos de prueba (ordenado e inverso).			
4	**Coherencia pseudocódigo–código:** el programa respeta la lógica del pseudocódigo propuesto y muestra el arreglo tras cada pasada, junto con los contadores.			
5	**Reflexión y buenas prácticas:** responde las preguntas de reflexión con argumentos, incluida la eficiencia, y el código usa nombres significativos, sangría y comentarios.			